Telematics Platform User Guide

Software Development Kit (SDK)

Document Information	
Filename	TP_F101U-00035_User_Guide_SDK
Status	Release
Revision Date	Nov 11, 2024
Version	03.00
Document ID	TP_F101U-00035

Standard Proprietary Terms
THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Table of contents

1 Introduction 4
1.1 Abstract 4
1.2 Purpose 4
1.3 Scope 4
1.4 Notational Conventions 4
2 Setup 5
2.1 Tools/Equipment Needed 5
2.1.1 Software for the sdk-constructor 5
2.1.2 Toolchain 6
3 Features/Components 7
3.1 Sdk-constructor 7
3.1.1 Overview 7
3.1.1.1 Adding packages 7
3.1.1.2 Image sizes 8
3.1.2 Setup 8
3.1.2.1 Tools/Equipment Needed 8
3.1.2.2 Setup Instructions 8
3.1.3 Usage 10
3.1.3.1 Basic Instructions 10
3.1.3.2 Detailed Instructions 10
3.1.4 Support of Integrity Validation Data —“UNECE” Feature 12
3.1.4.1 Background of Feature 12
3.1.4.2 Feature Description — SDK Generation with and without “UNECE” 12
3.1.5 Feature “Suspend Secure Boot” 18
3.1.5.1 Background and Basic Steps 18
3.1.5.2 Detailed Instructions to Activate “Suspend Secure Boot” 18
3.1.5.3 Disab“e "Suspend Secure B”ot" 19
3.1.6 Feature "Sign custom Trusted Application" 20
3.1.6.1 Background of the feature 20
3.1.6.2 Product expectations 20
3.1.6.3 Basic instructions 20
3.1.6.4 Compiling custom Trusted application 20
3.1.7 Examples 21
3.1.7.1 Collectd [FERMI_5G] 21
3.1.7.2 Nginx [FERMI_AP_4.5G] 25
3.1.7.3 Hello_World [BELL] 28
3.1.7.4 C++ conti-otp-test [WATSON] 30
3.1.7.5 Adding a Service [ALL] 33
3.1.8 Testing/Error Reporting 34

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.2 Linux Kernel using SDK 34
3.2.1 Compiling the Kernel source code 34
3.2.1.1 Downloading Linux Source Code 34
3.2.1.2 Applying conti patches to Kernel 35
3.2.1.3 Compilation 35
3.2.2 Kernel Module Compilation 35
3.2.2.1 Hello World Kernel Module Example 35
3.2.3 Flash new Kernel into target 38
3.2.3.1 Prerequisites 38
3.2.3.2 Perform required changes on Kernel 38
3.2.3.3 Generate new boot.img 38
3.2.3.4 Flash into target 38
3.2.3.5 Warnings 39
3.3 Increase system partition size. 39
3.3.1 Run resize_system.py script. 39
3.3.2 Warninig 40
3.3.3 Prerequisite 40
3.3.4 Additional software 40
4 Known Issues 40
5 FAQs 40
6 Troubleshooting/Tips and Tricks 40
7 References 40
7.1 Abbreviations/Acronyms/Definitions 40
7.2 Platform Documentation 40
7.3 Website links 41
8 **Revision History 42**

© Continental Automotive GmbH 2019
Confidential and Proprietary

1 Introduction

This document provides usage information on the Software Development Kit for the Telematics Platform.

1.1 Abstract

A Software Development Kit (SDK) is a set of tools that can be used to create and develop software.

The Telematics Platform SDK (OTP SDK) offers the option to create applications and integrate new applications into the final *File System (FS) Images.*

1.2 Purpose

This document serves as a User Manual for Telematics Platform sdk-constructor bundle.

1.3 Scope

This user guide provides details on how to use Telematics Platform sdk-constructor to build stand-alone applications on Linux based Automotive platforms such as ***TP_BELL*** or ***TP_FERMI***. It contains information that depicts the usage of APIs and demonstrate different use case scenarios.

This information is intended for software developers who will be using the Telematics SDK and assumes that the developers are familiar with Linux.

1.4 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	*A link to an external reference (e.g., website, document)*

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend

- Telematics Platform
- Product Specific
- 3rd Party

© Continental Automotive GmbH 2019
Confidential and Proprietary

2 Setup

2.1 Tools/Equipment Needed

2.1.1 Software for the sdk-constructor

Table 1 identifies the software needed by the sdk-constructor build environment to create images based on a specific platform baseline.

Table 1: Required Software

# OPERATING SYSTEM			
Item	***Item description***	***Version***	***Source/vendor***
1	Linux development workstation or virtual machine exceeding minimum desktop system requirements for running Ubuntu 64-bit OS	Ubuntu 16.04	Ubuntu Community/ Canonical, Ltd.
# Install basic tools			
apt-get update apt-get install -y apt-utils apt-get install -y vim apt-get install -y perl apt-get install -y python apt-get install -y sudo			
# OTP build system packages			
apt-get update apt-get install -y acl apt-get install -y autoconf apt-get install -y bc apt-get install -y bison apt-get install -y build-essential apt-get install -y dosfstools apt-get install -y doxygen apt-get install -y g++-multilib apt-get install -y gawk apt-get install -y gcc-multilib apt-get install -y kmod apt-get install -y lib32stdc++6 apt-get install -y lib32tinfo-dev apt-get install -y libgtk2.0-dev apt-get install -y libperl-dev			

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
apt-get install -y libxml-parser-perl
apt-get install -y lzop
apt-get install -y mscgen
apt-get install -y parted
apt-get install -y python3-setuptools
apt-get install -y scons
apt-get install -y texinfo
apt-get install -y udev
apt-get install -y xorriso
```

2.1.2 Toolchain

The toolchain is required for compiling the packages that will be added into the final image once the SDK constructor is executed.

Refer to the specific toolchain documentation for details on setup.

© Continental Automotive GmbH 2019
Confidential and Proprietary

3 Features/Components

The Telematics Platform SDK is formed by three elements:

- *tp_sdk_otp-<VARIANT>_pkg.zip*
- *tp_toolchain_otp-<VARIANT>_pkg.zip*
- *kernel_amend.tar.gz*

tp_sdk_otp-<VARIANT>_pkg.zip contents are

- *Documents*
- *Doxygen*
- *Drivers*
- *Readme.txt*
- *release*
  - *tz_images.tar.gz*
- *sdk*
  - *sdk-constructor.tar.gz*
  - *sdk.tar.gz*
  - *conti-signed-images.tar.gz*

tp_toolchain_otp-<VARIANT>_pkg.zip contents are

- *otp-host.tar.gz*
- *toolchain.tar.gz*

The release-toolchain contains the items required for the CROSS-COMPILING process (e.g., gcc) and the release-host is populated with all the packages required for create the final images (e.g., bison, automake, etc).

Toolchain setup, however, is responsibility of the party interested on the external packages to be added.

3.1 Sdk-constructor

3.1.1 Overview

The **sdk-constructor** is a subset of the build framework used to re-create images from an official release baseline.

Each sdk-constructor has a unique ID, which corresponds to the baseline name of the FS images that will be reproduced. For example, for a baseline *otp-mdm9x28-2.45.0.0*, there is a sdk-constructor that corresponds to that version (*otp-mdm9x28-sdk-constructor-2.45.0.0*).

The sdk-constructor contains all the configurations and items required to create a set of images with the addition of new external apps.

3.1.1.1 Adding packages

It is mandatory that the package must be in tar.gz format and must follow the structure for compilation like this:

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
project_dir
    Makefile
    /src
    /bin
    /inc
```

3.1.1.2 Image sizes

Select image sizes per the target HW configuration. Images can be either 2K or 4K format.

3.1.2 Setup

3.1.2.1 Tools/Equipment Needed

A Linux development environment for OTP must be available for setting up the SDK.

3.1.2.2 Setup Instructions

1. Unpack the **sdk-constructor.tar.gz** file into a specific location, this location will become our build environment, from here onwards, we'll refer to this path as our "***workspace***" and its environment variable will be $CAS_WORKDIR.

   **Note: The location of the workspace should not be a shared drive location, since there are issues related with unpacking tar files with symlinks.*

   **Tip: To untar the packages use the following command: tar -xvzf <NAME_PACKAGE>.tar.gz*

2. Untar **toolchain.tar.gz** file into the directory indicated in the workspace (refer to Figure 1 for the Constructor's Structure): ***./workspace/release-toolchain/fs/devel***

3. Untar **otp-host.tar.gz** tar file into the directory indicated in the workspace (refer to Figure 1 for the Constructor's Structure): ***./workspace/release-host/fs/host***

4. Untar **sdk.tar.gz** in the directory indicated in the workspace (refer to Figure 1 for the Constructor's Structure): ***./workspace/release/fs/devel***

5. Copy **conti-signed-images.tar.gz** to the workspace (refer to Figure 1 for the Constructor's Structure): ***./workspace***

6. Cd to the top directory of the workspace.

© Continental Automotive GmbH 2019
Confidential and Proprietary

7. Set the environment variables

```
export CAS_WORKDIR=<your_workarea_path> # e.g. export CAS_WORKDIR=$PWD
source ${CAS_WORKDIR}/.build/env-common
source ${CAS_WORKDIR}/.build/env-target
source ${CAS_WORKDIR}/.build/env-buildsys-static # This is required for image creation
export CC=${CROSS}gcc
export CXX=${CROSS}g++
```

Figure 1 shows an example of the work area ready to be used.

Figure 1: Constructor Structure/Work Area

```
File list                                          Description
------------                                       ------------------------------
.
├── .build
│   ├── env-buildsys-static
│   ├── env-common
│   └── env-target
├── .launchers
│   └── linux
│       └── sdk
│           ├── install-prodfs-sdk               # Entry point to generate images once all packages are decompressed
│           └── README                           # This README :)
├── project
│   └── mdm9x28
│       ├── config
│       │   ├── flash_all_via_qpst.xml
│       │   └── secbootcfg/*
│       └── scripts
│           ├── conti_sign_adapter.py
│           ├── create-image
│           ├── device_table.txt
│           └── __init__.py
├── release
│   ├── fs
│   │   ├── devel/                               # Untar of sdk.tar.gz (The complete one, from official release)
│   │   └── host -> ../../release-host/fs/host
│   └── images
│       └── devel/                               # 2K/ 4K folder will be placed with the images creation.
├── release-host/
│    └── fs
│        └── host                                # Untar of otp-host.tar.gz tar file
├── release-toolchain/
│    └── fs
│        └── devel                               # Untar of toolchain.tar.gz file
├── .repo
│   └── manifests
│       └── env-plf-config
├── sdk-constructor.tar.gz
├── conti-signed-images.tar.gz
└── tools
    └── build
        └── scripts/*
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.3 Usage

3.1.3.1 Basic Instructions

The bash script **install-prodfs-sdk** is responsible for creation of the **targets** folder, which contains the binaries that will be part of the final images.

This process is executed for the **release** and **release-toolchain** folder, and the list of binaries is taken from a manifest file inside of the tar files.

Once this process is completed, all the binaries (release and release-toolchain) will be copied into the **prodfs** folder.

3.1.3.2 Detailed Instructions

1. Setup the SDK workspace (refer to section 3.1.2.2)
2. Cd to the top directory of the workspace.
3. Once in the top directory of our workspace, execute install-prodfs-sdk:

```
"bash .launchers/linux/sdk/install-prodfs-sdk
     --workspace <workspace_path>
     --image_size <image_size>
     [--signed_images <path to conti-signed-images file>]
     [--build_version <build_vers>]
     [--trusted_app <TA_name> <path_to_TA>]"
```

Values for the named parameters:

`--workspace <workspace_path>`

mandatory. Must be the absolute path to the place where you unpacked the sdk-constructor.
Example: `${CAS_WORKDIR}`

`--image_size <image_size>`

mandatory. Depends on the project (refer to section 3.1.1.2 for the selection on the images sizes).

`[--signed_images <path to conti-signed-images file>]`

optional.
Parameter present = force use of signed images: the signed images of the delivery must be used.
The path to the signed images file is taken from the parameter.
"install-prodfs-sdk" will terminate with error if the file `<path to conti-signed-images>` is not there.
Example usage see below. See also section 3.1.2.2.

no/parameter omitted: signed images are used if present in the default location
`${CAS_WORKDIR}/conti-signed-images.tar.gz`.
If **"conti-signed-images.tar.gz"** is not found, the script proceeds with unsigned images from the sdk.

`[--build_version <build_vers>]`

© Continental Automotive GmbH 2019
Confidential and Proprietary

optional. Introduced for "UNECE" – see section 3.1.4 for details.

[--trusted_app <TA_name> <path_to_TA>]
optional.
<TA_name> – name of TA without extension
<path_to_TA> – absolute path to custom TA
Introduced for signing custom TA – see section 3.1.6 for details.

Example:

```
"bash .launchers/linux/sdk/install-prodfs-sdk
      --workspace ${CAS_WORKDIR}
      --image_size 4K
      --signed_images ${CAS_WORKDIR}/conti-signed-images.tar.gz"
```

Note: It's important to note that, depending on your **user permissions on your build environment, you might need to **login** as **root** to execute install-prodfs-sdk bash script.*

4. After execution:
   a. **"prodfs**" folder will be created in <workspace>/.build
   b. **"2K" or "4K"** folder will be created in <*workarea*>/release/images/devel, based on selected image size
   c. "**log**" folder will be populated: **`${CAS_WORKDIR}/log`** will contain a file **"summary.log"**. This log file is overwritten for each new script execution (no append).

5. Images will be created in <workspace>/release/images/devel/<image_size>/

© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.4 Support of Integrity Validation Data —“UNECE” Feature

3.1.4.1 Background of Feature

With the “UNECE” feature, ConMod / OTP platform introduces “Support of Integrity Validation Data” by implementing a mechanism to add a version number to binary images.

This version number can be e.g.

- a gerrit hash commit taken from the gerrit project this image is built from,
- a general “build version”.

3.1.4.2 Feature Description — SDK Generation with and without “UNECE”

3.1.4.2.1 Affected Images

Image File	Name
`abl.elf`	`abl`
`aop.mbn`	`aop`
`apdp.mbn`	`apdp`
`boot.img`	`boot`
`devcfg.mbn`	`tz_devcfg`
`dsp2.ubifs`	`dsp2`
`hyp.mbn`	`qhee`
`multi_image.mbn`	`multi_image`

`sbl1.mbn`
`sbl1-audi-conmod.mbn`
`sbl1-chestnut.mbn`
	`sbl`

`system.ubifs`	`system`
`tz.mbn`	`tz`
`uefi.elf`	`uefi`
`xbl_cfg.elf`	`xbl_cfg`

3.1.4.2.2 CRC and Version

3.1.4.2.2.1 Without “UNECE” Feature

During generation of these images (`.mbn`, `.elf`, `.img`, `.ubifs`), the CRC is added to the image footer. No version available.

3.1.4.2.2.2 With “UNECE” Feature

- the CRC is no longer contained in the image footer,
- a “version” is added to the image footer,

© Continental Automotive GmbH 2019
Confidential and Proprietary

- the CRC of the image needs to be calculated dynamically → 4-byte CRC value for all supported images is calculated using the "**ubicrc32**" utility.

3.1.4.2.3 Image Generation Procedure / SDK Generator Call

Images are created by the script "`create-image`" contained in the "`sdk-constructor`" tarball of the SDK package in:

```
<SDK>/tp_sdk_otp-sa515m-thick-3.y_pkg/sdk/
   sdk-constructor.tar:
     /.build/scripts
```

"`create-image`" is called by the "main" SDK constructor script "`install-prodfs-sdk`", which is located in the "`sdk-constructor`" tarball in:

```
<SDK>/tp_sdk_otp-sa515m-thick-3.y_pkg/sdk/
   sdk-constructor.tar:
     /.launchers/linux/sdk
```

The SDK generator is invoked by "`install-prodfs-sdk`". With introduction of the UNECE feature, this script received an optional parameter. Examples:

```
# SDK gen. before UNECE introduction or intending to use default build version:
bash .launchers/linux/sdk/install-prodfs-sdk –workspace ${CAS_WORKDIR} –image_size 4K

# SDK gen. with UNECE feature, setting explicit version number for "system.ubifs":
bash .launchers/linux/sdk/install-prodfs-sdk –workspace ${CAS_WORKDIR} –image_size 4K
--build_version otp-sa515m-thick-3.1.16.21
```

The result depends on whether UNECE is present or not:

3.1.4.2.3.1 Without "UNECE" Feature

The script "`create-image`" generates all images in **2K/4K** folder each time it is run and adds the CRC to the image footer. No version information contained in images.

3.1.4.2.3.2 With "UNECE" Feature

With the "UNECE" feature in place, the modified script

- Generates the images from the provided binaries **ONCE** — thus "filling" the SDK workarea's **2K/4K** folder. If an image is already available, it will not be modified any more ⚠.
  Exception: "`system.ubifs`" – see next section.
- Appends the version number to the image footer. Version number used:

© Continental Automotive GmbH 2019
Confidential and Proprietary

  - for "`system.ubifs`": part of Linux environment variable "`bld_version_number`" (= value of optional parameter "–`build_version`" of "`install-prodfs-sdk`" or a default build version) – see next section ⚠ .
  - git hash commits for the others.

- ⚠ Creates / appends a line to a file "`image_crc_versions.csv`" in the **2K/4K** folder, with info on this new image:
  - Image name
  - Image type
  - Image CRC (dynamically calculated, **NOT** written to image but only to this csv file)
  - Image Version

```
<Image Name>                          <Image Type>     <Image CRC>   <Image Version>
abl.elf                               PROGRAM_TYPE     0xb2ad2370    1285ed
aop.mbn                               PROGRAM_TYPE     0x898c379d    028e95
apdp.mbn                              PROGRAM_TYPE     0xf265183c    5509a7
boot.img                              PROGRAM_TYPE     0xd67c2e9c    f2ea7b
devcfg.mbn                            PROGRAM_TYPE     0x224fedf0    1e05fa
dsp2.ubifs                            PROGRAM_TYPE     0xb113bae3    1.13.07
hyp.mbn                               PROGRAM_TYPE     0xdb2c30d5    1e05fa
multi_image.mbn                       PROGRAM_TYPE     0xe745cdb1    603602
sbl1-audi-conmod.mbn                  PROGRAM_TYPE     0x2e9be72b    a5c319
sbl1-chestnut.mbn                     PROGRAM_TYPE     0x653f14fa    a5c319
sbl1.mbn                              PROGRAM_TYPE     0xa140e6c6    a5c319
system.ubifs                          PROGRAM_TYPE     0xe77b1b68    3.1.16.21     (!!!!!!!)
tz.mbn                                PROGRAM_TYPE     0x1601928d    1e05fa
uefi.elf                              PROGRAM_TYPE     0x48087f63    31e8fa
xbl_cfg.elf                           PROGRAM_TYPE     0xd8153069    a5c31
```

3.1.4.2.4 "system.ubifs" Image and Version with "UNECE"

- A "general" build version number is added to the footer of "`system.ubifs`".
- The SDK offers an interface to control this build version, thus enabling a user to specify his own build / system version to be used as version of "`system.ubifs`".
- This can be controlled by specifying the optional parameter –`build_version`" of "`install-prodfs-sdk`".
- This sets the Linux environment variable "`bld_version_number`" before SDK generation.
- The nomenclature of this "build version number" should follow

  `<dash separated prefix>-`**`<dot separated desired version>`**`-<dash separated suffix>`

  The script will use the `<dot separated desired version>` as image version.

- Steps to be executed:

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
# Explicitly specify the optional version number parameter
bash .launchers/linux/sdk/install-prodfs-sdk –workspace ${CAS_WORKDIR} –image_size  4K –
build_version ConMod-Y.115.0.1-devel
```

- Then "`create-image`" will add version **Y.115.0.1** to "`system.ubifs`" and update the content of "`image_crc_versions.csv`" with this info automatically.
- If the parameter "build version" is not specified, the SDK generator uses its "internal" build version as default (which is related to the baseline / SW version, e.g. "`otp-sa515m-thick-3.1.16.21`"):

```
# Omit the optional version number parameter -> default
bash .launchers/linux/sdk/install-prodfs-sdk –workspace ${CAS_WORKDIR} –image_size 4K
```

- In this case, "`create-image`" would add version **3.1.16.21** to "`system.ubifs`" and "`image_crc_versions.csv`".

Every time the SDK creator / "`install-prodfs-sdk`" is re-run, the script will

- not modify images other than "`system.ubifs`",
- re-evaluate the value of "`bld_version_number`" (which either contains the specified **build_vers** parameter or the default),
- re-write the version of "`system.ubifs`" with the relevant info, and update "`image_crc_versions.csv`".

I.e. by a re-run of "`install-prodfs-sdk`" as specified above, it is possible to override the version of "`system.ubifs`" with any desired version number later, as often as desired.

3.1.4.2.5 How to Identify the "UNECE" Feature

How can you identify whether or not the SDK you received contains the "UNECE" feature implementation?

1. "`install-prodfs-sdk`" accepts the parameter "[**`--build_version <build_vers>`**]"
2. After SDK generation, look for the file in the **2K/4K** folder, e.g.

```
${CAS_WORKDIR}/release/images/devel/4K/     image_crc_versions.csv
```

- File present: SDK contains the feature. Applicable images have the version in the footer, no CRC.
- File not present: SDK does not have the UNECE feature. Then the images have the CRC in their footer and the script "`swl_verify_images_version_crc`" is not available either → see next section.

And if the file is present, but you did not (yet) specify your own "build version number" as version of "`system.ubifs`" – no problem, just re-run the SDK generation with the parameter "**build_vers**" as specified above.

3.1.4.2.6 How to get CRC and Version in SDK with "UNECE"

3.1.4.2.6.1 Host Environment

© Continental Automotive GmbH 2019
Confidential and Proprietary

The info is contained in "`image_crc_versions.csv`" in the **2K/4K** folder. The 4-byte CRC is calculated using "**ubicrc32**".

3.1.4.2.6.2 Target Environment

A script "`swl_verify_images_version_crc`" is provided with the UNECE feature in the "`sdk`" tarball of the SDK package:

```
<SDK>/tp_sdk_otp-sa515m-thick-3.y_pkg/sdk/
   sdk.tar:
     /tst/bin
```

This is flashed to the target under

`/tst/bin/`

Calling this script provides the same information that is contained in "`image_crc_versions.csv`" (plus the info on the Modem Config "`mcm_`" images). The script also uses "**ubicrc32**" to calculate the 4-byte CRC:

```
[root@sa515m ~]# /tst/bin/swl_verify_images_version_crc

<Image Name>                                    <Image Type>    <Image CRC>     <Image Version>

abl                                             PROGRAM         0x1caf8adb      eb3724a
aop                                             PROGRAM         0xde7a5fa3      c028e95
apdp                                            PROGRAM         0x8da98acb      c028e95
boot                                            PROGRAM         0xd587e93c      f53188a
dsp2                                            PROGRAM         0xf3882be8      2fae441
mcfg_hw_FE515_LE_SS.mbn                         CONFIGURATION   0xe745cdb1      a01800c
mcfg_sw_AMX_Commercial_MX.mbn                   CONFIGURATION   0x4h3745c1      a011600
mcfg_sw_ATT_VoLTE.mbn                           CONFIGURATION   0x6382cdb1      a020335
mcfg_sw_Brazil_Commercial.mbn                   CONFIGURATION   0xeab09db1      a012100
mcfg_sw_CMCC_Commercial_Volte_OpenMkt.mbn       CONFIGURATION   0xc9820db1      a012010
mcfg_sw_Cubic_VoLTE.mbn                         CONFIGURATION   0xa9f72cd1      a01fa01
mcfg_sw_DCM_Commercial.mbn                      CONFIGURATION   0x298745b1      a010d0d
mcfg_sw_DT_VoLTE_Commercial.mbn                 CONFIGURATION   0x789243db      a011f1f
mcfg_sw_KT_Commercial_KT_LTE.mbn                CONFIGURATION   0x98sacd34      a01280b
mcfg_sw_ROW_Commercial.mbn                      CONFIGURATION   0xdac5cd45      a010809
mcfg_sw_Telstra_Commercial.mbn                  CONFIGURATION   0xbadefee0      a010f00
mcfg_sw_Verizon_CDMAless.mbn                    CONFIGURATION   0x0effeff0      a010126
mcfg_sw_Vodafone_M2M_Commercial_Global.mbn      CONFIGURATION   0x987decab      a010400
mcfg_sw_Vodafone_VoLTE_Germany.mbn              CONFIGURATION   0x98sebcaf      a010449
mcfg_sw_Vodafone_VoLTE_Turkey.mbn               CONFIGURATION   0xdadadada      a0104c6
multi_image                                     PROGRAM         0xf873ca30      9eafe72
qhee                                            PROGRAM         0x5eaac1d6      f1e05fa
sbl                                             PROGRAM         0xeff02f7e      fa5c319
system                                          PROGRAM         0x4aecaf06      3.1.16.21
tz                                              PROGRAM         0x36466040      f1e05fa
tz_devcfg                                       PROGRAM         0x01984278      f1e05fa
uefi                                            PROGRAM         0x4174213e      831e8fa
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
xbl_config                                    PROGRAM         0xd8153069      fa5c319
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.5 Feature "Suspend Secure Boot"

3.1.5.1 Background and Basic Steps

"Secure Boot" is enabled through a set of hardware fuses. Code images to be executed must be signed by the trusted authority identified in the hardware fuses. Additionally, the `sec.elf` image must have been flashed onto the `sec` partition. Once Secure Boot has been activated, it is usually no longer possible to run / boot unsigned SW images.

The "***Suspend Secure Boot***" feature offers a way to still operate a device with unsigned application and kernel images, even though the hardware fuses have already been initialized for Secure Boot. The feature suspends the secure boot sequence starting from APPSBOOT.

The feature is enabled by

- modifying the debug policy config in the SDK,
- (re)generating and signing the "`apdp.mbn`" image,
- flashing "`apdp.mbn`" to the system.

3.1.5.2 Detailed Instructions to Activate "Suspend Secure Boot"

The following steps shall be executed after

- unpacking "`sdk.tar.gz`", "`sdk-constructor.tar.gz`" and "`conti-signed-images.tar.gz`" to the workspace
- setting up the SDK's "`prodfs`" folder with "**`install-prodfs-sdk`**"

(for details see section 3.1.2 "Setup").

3.1.5.2.1 Modify Debug Policy

The debugpolicy configuration "`sdx55_debugpolicy.xml`" is delivered with the "**`sdk-constructor`**" tarball:

```
sdk-constructor.tar.gz:
   /.build/config/secbootcfg/debugpolicy/sdx55_debugpolicy.xml
```

After unpacking, `sdk-constructor.tar.gz`, the following modifications must be done in this file:

<u>Add the serial number of your device:</u>

```
    …
    </metadata>
<serial_num_list> <!—Range: 1 to 200 →
   <serial_num>0x2466e4ca</serial_num>
   <serial_num>0x2466e4cb</serial_num>
</serial_num_list>
    …
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

The device serial number can be found in the output of t“e "`adb devi”es`" command. It is possible to specify up to 200 unique serial numbers in this list field. All corresponding devices are then supported by the Suspend Secure Boot. I.e. it is possible to (re)use this debugpolicy file for all listed devices.

Set parameter `DP_ENABLE_SUSPEND_SECUREBOOT_OEM` value to 1:

```
<flag> <bit_pos>60</bit-pos> <value>1</value>
   </flag>—!--DP_ENABLE_SUSPEND_SECUREBOOT_OEM-->
```

Optionally, you can also activate any other development features by setting the corresponding `<flag>` to **1**.

3.1.5.2.2 (Re)Create t“e "`apdp.”bn`" Signed Image

(Re)Create and si“n "`apdp.”bn`" with the following commands:

```
export CAS_WORKDIR=<YOUR_SDK_WORKSPACE>
source .build/env-buildsys-static
source ${bld_build_dir}/env-common
source ${bld_build_dir}/env-target
source ${bld_script_dir}/misc/bootstrap
source ${bld_script_dir}/misc/log
./tools/build/scripts/secboot-image-si‘n 's’g‘' 'debugpol’cy'
```

➔ Signed `apdp.mbn` image is created in folder
`<workarea>/release/images/devel/`**`debugpolicy`**`/`

3.1.5.2.3 Fla“h "`apdp.”bn`" to enable Suspend Secure Boot

- Copy signed image `apdp.mbn` from `/debugpolicy/` image folder.
- Flash `apdp.mbn` either via fastboot or software download package.

  Example for fastboo `fastboot flash apdp apdp.mbn`
  `fastboot reboot`

3.1.5.3 Disab“e "Suspend Secure B”ot"

- Use fastboot to erase **apdp**:

```
fastboot erase apdp
fastboot reboot
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

3.1.6 Feature "Sign custom Trusted Application"

3.1.6.1 Background of the feature

This feature allows to sign a custom Trusted Application which is loaded and operates in Trust Zone. For production units it is necessary to have custom TA properly signed by QC mechanism for successful verification.

3.1.6.2 Product expectations

Users of this guide are responsible to create image signing adapter at the path `.build/scripts/conti_sign_adapter.py` and define the class *contiSignAdapter* and implement the method `sign_cust_ta` for signing custom trusted application.

Check SecureBoot 5G NAD ConMod User_Guide (TP_F101U-00054) for more details on signing procedure.

3.1.6.3 Basic instructions

This is an optional feature that is triggered under SDK environment and enabled by providing additional parameters to **`install-prodfs-sdk`** script. Additional parameters are custom Trusted Application name (i.e. TA_name) and absolute path to it (i.e. TA_path), see section 3.1.3.2

Example:

```
"bash .launchers/linux/sdk/install-prodfs-sdk
      --workspace ${CAS_WORKDIR}
      --image_size 4K
      --signed_images ${CAS_WORKDIR}/conti-signed-images.tar.gz
      --trusted_app ${TA_name} ${TA_path}"
```

TA_name should be without extension. If the full name of custom TA is “customTA.mbn”, then TA_name variable should be “custom_TA”. After script execution, signed custom TA will be placed under path *${TA_path}/signed/*.

3.1.6.4 Compiling custom Trusted application

SDK provides possibility to compile custom Trusted application, as well as sample code for creating it.

The environment for the compilation of custom TA is in tz_images.tar.gz, which is part of tp_sdk_otp-<VARIANT>_pkg.zip, see section 3 Features/Components.

Source code for custom TA should be placed in *trustzone_images/ssg/bsp/trustzone/qsapps/${TA_name}/* folder. *trustzone_images/ssg/bsp/trustzone/qsapps/sampleapp* is a sample code that can be used as a reference.

Custom TA compilation can be done with the following steps:

© Continental Automotive GmbH 2019
Confidential and Proprietary

1. Set environmental variables for Qualcomm LLVM compiler (4.0.17 is used as a reference):

```
declare   -x   LLVM32INC="/home/uic75888/proj/eso/workdir-eso/Snapdragon-llvm-4.0.17-linux64/armv7-none-eabi/libc/include"
declare    -x    LLVMBIN="/home/uic75888/proj/eso/workdir-eso/Snapdragon-llvm-4.0.17-linux64/bin"
declare   -x   LLVMCLANG="/home/uic75888/proj/eso/workdir-eso/Snapdragon-llvm-4.0.17-linux64/lib/clang/4.0.17"
declare    -x    LLVMINC="/home/uic75888/proj/eso/workdir-eso/Snapdragon-llvm-4.0.17-linux64/aarch64-none-elf/libc/include"
declare    -x    LLVMLIB="/home/uic75888/proj/eso/workdir-eso/Snapdragon-llvm-4.0.17-linux64/lib/clang/4.0.17/lib/linux"
declare  -x  LLVMLIBPROP="/home/uic75888/proj/eso/workdir-eso/Snapdragon-llvm-4.0.17-linux64/lib/clang/4.0.17/lib/linux-propri_rt"
declare   -x   LLVMROOT="/home/uic75888/proj/eso/workdir-eso/Snapdragon-llvm-4.0.17-linux64"
```

2. Run compilation command:
   `python build_all.py CHIPSET=sdx55 -b TZ.XF.5.0 --cbt=TA_name`

3. Compiled custom TA is located under path *trustzone_images/build/ms/bin/EATAANBA/*.

3.1.7 Examples

3.1.7.1 Collectd [FERMI_5G]

collectd is a daemon which collects system and application performance metrics periodically and provides mechanisms to store the values in a variety of ways (https://collectd.org/). This example shows how to add collectd into an OTP image.

1. In your workarea ($CAS_WORKDIR), download the collectd package (version 7.5.2).
   Get the collectd folder in your working directory (**$CAS_WORKDIR/collectd**)

2. Inside of your **collectd** folder, execute the following commands:
   a) autoreconf:

```
${CAS_WORKDIR}/release-host/fs/host/usr/bin/autoreconf --install --symlink --force
```

   b) Configure

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
have_swapctl=no CFLAGS+=" -Wno-format-truncation" ./configure
    --build=${CAS_HOST}
    --host=${CAS_TARGET}
    --target=${CAS_TARGET}
    --prefix=/usr
    --exec_prefix=/usr
    --bindir=/usr/bin
    --sbindir=/usr/sbin
    --libexecdir=/usr/libexec
    --datadir=/usr/share
    --sysconfdir=/etc
    --sharedstatedir=/com
    --localstatedir=/var
    --libdir=/usr/lib
    --includedir=/usr/include
    --oldincludedir=/usr/include
    --infodir=/usr/share/info
    --mandir=/usr/share/man
    --disable-static
    --with-fp-layout=nothing
    --disable-java
    --disable-perl
    --with-libperl=no
    --with-perl-bindings=no
    --disable-notify_desktop
    --disable-network
    --enable-syslog
    --enable-csv
```

c) Run “make”:

```
make
```

d) Run “make” install

```
make install DESTDIR=${CAS_WORKDIR}/collectd_exec
```

3. For this application, a Service Unit is required. Service units end with the “**.service”** file extension and serve a similar purpose as init scripts. The **nginx.service** file should be created as the following example:

```
${CAS_WORKDIR}/collectd_exec/collectd.service
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
[Unit]
Description=Collectd
After=local-fs.target network.target
Requires=local-fs.target network.target

[Service]
ExecStart=/usr/sbin/collectd -C /etc/collectd.conf -f
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

4. Some changes should be made to the files generated before copy them to the targe:

```
sed -i 's!^#FQDNLookup[ \t]*true!FQDNLookup   false!g' ${CAS_WORKDIR}/collectd_exec/etc/col-
lectd.conf
rmdir ${CAS_WORKDIR}/collectd_exec/var/run
rmdir --ignore-fail-on-non-empty ${CAS_WORKDIR}/collectd_exec/var
```

5. Create the **"**external_packages**"** folder.

```
mkdir -p ${CAS_WORKDIR}/external_packages
```

6. Copy your generated files into the “external_packages**”** folder with the corresponding directory structure in the target:

```
cp ${CAS_WORKDIR}/<path_to_binary>/<binary> ${CAS_WORKDIR}/external_packages/usr/bin/
```

***NOTE**: The structure should be created manually before use "**cp**" for the copy. Other option is to use **rsync** (for example, "rsync -ravl --no-o --no-g")*

© Continental Automotive GmbH 2019
Confidential and Proprietary

7. The Executable binary and other files required by the service should be copied to the final target:

```
cp -a ${CAS_WORKDIR}/collectd_exec/collectd.service ${CAS_WORKDIR}/external_packages/lib/systemd/system/
cp -a ${CAS_WORKDIR}/collectd_exec/etc/collectd.conf ${CAS_WORKDIR}/external_packages/etc/
cp -a ${CAS_WORKDIR}/collectd_exec/usr/lib/collectd/*.so ${CAS_WORKDIR}/external_packages/usr/lib/collectd/
cp -a ${CAS_WORKDIR}/collectd_exec/usr/sbin/* ${CAS_WORKDIR}/external_packages/usr/sbin/
cp -a ${CAS_WORKDIR}/collectd_exec/usr/share/collectd/* ${CAS_WORKDIR}/external_packages/usr/share/collectd
```

8. Finally, the new set of images can be created:

```
bash .launchers/linux/sdk/install-prodfs-sdk ${CAS_WORKDIR} <2K or 4K>
```

9. Once the board been flashed with the images generated, you can check the status of the **collectd.service**. The following image shows the expected result:

```
[  OK  ] Reached target Network.
[  OK  ] Started Collectd.
         Starting Container machines...
         Starting The NGINX HTTP and reverse proxy server...
         Starting OpenBSD Secure Shell server...
[FAILED] Failed to start Container machines.
See 'systemctl status systemd-nspawn@machines.service' for details.
[  OK  ] Started The NGINX HTTP and reverse proxy server.
[FAILED] Failed to start OpenBSD Secure Shell server.
See 'systemctl status sshd.service' for details.
[  OK  ] Found device /dev/ttyDebugConsole.
[  OK  ] Started Serial Getty on ttyDebugConsole.
[  OK  ] Reached target Mid-boot services.
         Starting Location Manager Service...
         Starting Persistency Admin Service...
[  OK  ] Reached target Login Prompts.
[  OK  ] Reached target Containers.
[    8.542409] (null): mxc_isi_capture_open, No remote pad found!
[    8.548083] (null): mxc_isi_capture_open, No remote pad found!
[    8.567211] (null): mxc_isi_capture_open, No remote pad found!
[    8.577958] (null): mxc_isi_capture_open, No remote pad found!
[  OK  ] Stopped OpenBSD Secure Shell server.
         Starting OpenBSD Secure Shell server...
[FAILED] Failed to start OpenBSD Secure Shell server.
See 'systemctl status sshd.service' for details.
[  OK  ] Started Persistency Admin Service.
[  OK  ] Started Location Manager Service.
[  OK  ] Reached target Sound Card.
[  OK  ] Reached target Late-boot services.
[  OK  ] Reached target Multi-User System.
[  OK  ] Reached target Graphical Interface.
         Starting Update UTMP about System Runlevel Changes...
[    8.875120] IPv6: ADDRCONF(NETDEV_UP): usb0: link is not ready
[  OK  ] Started Update UTMP about System Runlevel Changes.
[  OK  ] Stopped OpenBSD Secure Shell server.
         Starting OpenBSD Secure Shell server...
[FAILED] Failed to start OpenBSD Secure Shell server.
See 'systemctl status sshd.service' for details.

##################################################################
# Copyright (C) 2019 Continental Automotive Systems, Inc.
#
#
# Platform Version: OTP-IMX8-MEK-3.Y_JENKINS_82-DEVEL
# Linux Version:    4.14.62-imx_4.14.62_1.0.0_beta-g3d7f639
# Build Date:       Tue May 14 07:31:37 UTC 2019
#
# WARNING! Development: Console always allowed
#          (DO NOT PROVIDE IMAGE TO OEM!)
##################################################################

imx8 login: root
[root@imx8 ~]# systemctl status collectd
[[0;1;32m●[[0m collectd.service - Collectd
   Loaded: loaded (/lib/systemd/system/collectd.service; enabled; vendor preset:
 enabled)
   Active: [[0;1;32mactive (running)[[0m since Tue 2019-05-14 06:59:56 UTC; 1min
 42s ago
 Main PID: 3643 (collectd)
    Tasks: 11 (limit: 2027)
   CGroup: /system.slice/collectd.service
           └─3643 /usr/sbin/collectd -C /etc/collectd.conf -f
```

notive GmbH 2019
tial and Proprietary

3.1.7.2 Nginx [FERMI_AP_4.5G]

This example shows how to add nginx to an OTP image for the imx8. Nginx is a free open source web server focused on high performance, high concurrency and low memory usage (www.nginx.com).

1. In your workarea ($CAS_WORKDIR), download the nginx package (e.g. version 1.13.5).
   Get the nginx folder in your working directory: $CAS_WORKDIR/nginx

2. Inside of your **nginx** folder, execute the following commands:

   e) Set ptrsize (specific for this example package):

```
export PTRSIZE=4
```

   f) Configure

```
CC=${CROSS}gcc ./auto/configure \
--crossbuild=Linux:${CAS_ARCH}:CROSS \
--user=nobody --group=nobody \
--with-endian=big --with-int=4 \
--with-long=${PTRSIZE} \
 --with-long-long=8 \
--with-ptr-size=${PTRSIZE} \
--with-sig-atomic-t=${PTRSIZE} \
--with-size-t=${PTRSIZE} \
--with-off-t=${PTRSIZE} \
--with-time-t=${PTRSIZE} \
--with-sys-nerr=132 \
--http-log-path=/var/log/nginx/access.log \
--error-log-path=/var/log/nginx/error.log \
--http-client-body-temp-path=/run/nginx/client_body_temp \
--http-proxy-temp-path=/run/nginx/proxy_temp \
--http-fastcgi-temp-path=/run/nginx/fastcgi_temp \
--http-uwsgi-temp-path=/run/nginx/uwsgi_temp \
--http-scgi-temp-path=/run/nginx/scgi_temp \
--pid-path=/run/nginx/nginx.pid --prefix=/usr \
--conf-path=/etc/nginx/nginx.conf \
--without-http_rewrite_module \
--without-http_proxy_module \
--without-mail_pop3_module \
 --without-mail_imap_module \
--without-mail_smtp_module \
 --without-http_gzip_module \
```

   Logs: nginx_1.13.5_configure

   g) Run make:

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
make
```

nginx_1.13.5_make

Logs:

h) Run Make install

```
make install DESTDIR=${CAS_WORKDIR}/nginx_exec
```

nginx_1.13.5_make_install

Logs:

3. Once the execution process (**step 2**) is completed, you will find the binary in the following path:

```
${CAS_WORKDIR}/nginx_exec/usr/sbin/nginx
```

4. For this application, a Service Unit is required. Service units end with the "**.service"** file extension and serve a similar purpose as init scripts. The **nginx.service** file should be created as the following example:

${CAS_WORKDIR}/nginx_exec/nginx.service

```
[Unit]
Description=The NGINX HTTP and reverse proxy server
After=syslog.target network.target remote-fs.target nss-lookup.target

[Service]
Type=forking
PIDFile=/run/nginx/nginx.pid
ExecStart=/usr/sbin/nginx
ExecStop=/usr/sbin/nginx -s stop
ExecReload=/usr/sbin/nginx -s reload

[Install]
WantedBy=multi-user.target
```

5. The Executable binary and other files required by the service should be copied to the final target:

```
cp ${CAS_WORKDIR}/nginx_exec/usr/sbin/nginx ${CAS_WORKDIR}/external_packages/usr/sbin/
                                                    # Executable Binary
cp ${CAS_WORKDIR}/nginx_exec/nginx.service
${CAS_WORKDIR}/external_packages/lib/systemd/system/         # Service Unit

cp -r ${CAS_WORKDIR}/nginx_exec/usr/html/*
${CAS_WORKDIR}/external_packages/usr/                        # index.html
cp -r ${CAS_WORKDIR}/nginx_exec/etc/nginx/*
${CAS_WORKDIR}/external_packages/etc/nginx/                  # Configs
```

6. **nginx service** will require some modifications for logs and run paths:

© Continental Automotive GmbH 2019
Confidential and Proprietary

a) **nginx.conf**

```
echo "d /run/nginx - - - -" > ${CAS_WORKDIR}/external_packages/etc/tmpfiles.d/nginx.conf
echo "d /var/log/nginx 0755 root root -" >>
${CAS_WORKDIR}/external_packages/etc/tmpfiles.d/nginx.conf
```

b) **logs**

```
mkdir -p ${CAS_WORKDIR}/external_packages/var/log/nginx
mkdir -p ${CAS_WORKDIR}/external_packages/etc/nginx
cd ${CAS_WORKDIR}/external_packages/etc/nginx && ln -snf /var/run/nginx run
```

7. Finally, the new set of images can be created:

```
bash .launchers/linux/sdk/install-prodfs-sdk ${CAS_WORKDIR} <2K or 4K>
```

8. Once the board been flashed with the new images, you can check the status of the **nginx.service**. The following image shows the expected result:

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
         Starting OpenBSD Secure Shell server...
         Starting The NGINX HTTP and reverse proxy server...
         Starting Container machines...
[  OK  ] Started OpenBSD Secure Shell server.
[FAILED] Failed to start Container machines.
See 'systemctl status systemd-nspawn@machines.service' for details.
[  OK  ] Started The NGINX HTTP and reverse proxy server.
[  OK  ] Started Location Manager Service.
[  OK  ] Started Persistency Admin Service.
[  OK  ] Reached target Late-boot services.
[  OK  ] Reached target Containers.
[    9.320318] (null): mxc_isi_capture_open, No remote pad found!
[    9.321308] (null): mxc_isi_capture_open, No remote pad found!
[    9.333033] (null): mxc_isi_capture_open, No remote pad found!
[    9.338255] (null): mxc_isi_capture_open, No remote pad found!
[  OK  ] Reached target Multi-User System.
[  OK  ] Reached target Graphical Interface.
         Starting Update UTMP about System Runlevel Changes...
[  OK  ] Started Update UTMP about System Runlevel Changes.
[  OK  ] Reached target Sound Card.
[    9.603947] IPv6: ADDRCONF(NETDEV_UP): usb0: link is not ready


##################################################################
# Copyright (C) 2019 Continental Automotive Systems, Inc.
#
#
# Platform Version: REFS/HEADS/OTP-PILOT-3.Y-DEVEL
# Linux Version:    4.14.62-imx_4.14.62_1.0.0_beta-g3d7f6399c2a6
# Build Date:       Wed May 15 20:24:39 CDT 2019
#
# WARNING! Development: Console always allowed
#           (DO NOT PROVIDE IMAGE TO OEM!)
##################################################################

imx8 login:
[root@imx8 ~]# systemctl status nginx

[[0;1;32m•[[0m nginx.service - The NGINX HTTP and reverse proxy server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor preset: en
abled)
   Active: [[0;1;32mactive (running)[[0m since Wed 2019-05-15 03:14:50 UTC; 42s
ago
  Process: 3650 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
 Main PID: 3661 (nginx)
    Tasks: 2 (limit: 2027)
   CGroup: /system.slice/nginx.service
           ├─3661 nginx: master process /usr/sbin/nginx
           └─3664 nginx: worker process
```

Log: nginx_running_on_board.txt

3.1.7.3 Hello_World [BELL]

The following steps will help you to build and install a basic example of a "Hello World" application created through the SDK Suite.

Hello_World_TP.tar.gz

1. Download the “Hello World” tar file:

© Continental Automotive GmbH 2019
Confidential and Proprietary

2. Create a directory named **package** in the workspace:

```
**mkdir package # e.g. ${CAS_WORKDIR}/package
```

3. Extract your "Hello World" tar package:

```
**cd package
tar -zxvf Hello_World_TP.tar.gz
```

4. Set the environment variables:

```
export CAS_WORKDIR=<your_workarea_path> # e.g. export CAS_WORKDIR=$PWD
source ${CAS_WORKDIR}/.build/env-common
source ${CAS_WORKDIR}/.build/env-target
source ${CAS_WORKDIR}/.build/env-buildsys-static # This is required for image creation
export CC=${CROSS}gcc
export CXX=${CROSS}g++
```

5. Create the "external_packages" folder at the ${CAS_WORKDIR} level.

```
mkdir -p ${CAS_WORKDIR}/external_packages
```

6. Compile the application:

```
**cd <your-package> # e.g. /<CAS_WORKDIR>/package/Hello_World_TP
make
```

Log: Hello_World_Example_Log.txt

7. Copy your generated files into the external_packages folder with the corresponding directory structure for the target:

```
cp ${CAS_WORKDIR}/<path_to_binary>/<binary> ${CAS_WORKDIR}/external_packages/usr/bin/
```

8. Execute install-prodfs-sdk for the image creation:

```
bash .launchers/linux/sdk/install-prodfs-sdk --workspace ${CAS_WORKDIR}
--image_size <2K or 4K> [--signed_images  <path to conti-signed-images file>]
[--build_version <build_vers>]
```

9. The generated images will be located under the 2K or 4K folder (e.g. `${CAS_WORKDIR}`/release/images/devel/2K), replacing the images generated before.
10. Flash the images on the board

11. Once the images are flashed to the board and the board is booted, the binary can be run from a terminal, e.g:

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
[root@mdm9x28 bin]# hello-world
```

12. The output would be directed to the /var/log/messages file.

```
Apr  6 03:17:40 mdm9x28 user.debug hello-world: main:Hello World!
```

3.1.7.4 C++ conti-otp-test [WATSON]

The **conti-otp-test** package contains an internal OTP testing application written in C++ (internal only).

This package provides a mechanism to test the APIs in a consolidated way through only one application. For this example, the Software Version used is **otp-mdm9x28-2.50.2.1 Release.**

The following steps can be used for compile conti-otp-test via SDK Suite and generate a new set of images with it:

1. Download the **conti-otp-test** source code.

```
git clone buic-scm:p1/package/conti/conti-otp-test
```

The workarea should looks like this:

```
|-- .build
|-- .repo
|-- build.sh
|-- conti-otp-test
|-- logs
|-- project
|-- release
|-- release-host
|-- release-toolchain
`-- tools
```

2. **Environment -** The following commands will help to set all the variables into the environment and make it be ready for any compiling.

```
export CAS_WORKDIR=<your_workarea_path> # e.g. export CAS_WORKDIR=$PWD
source ${CAS_WORKDIR}/.build/env-common
source ${CAS_WORKDIR}/.build/env-target
source ${CAS_WORKDIR}/.build/env-buildsys-static # This is required for image creation
export CC=${CROSS}gcc
export CXX=${CROSS}g++
```

3. **Compiling -** Go into the package repository: ${**CAS_WORKDIR}/conti-otp-test** and run "make"

```
$ cd ${CAS_WORKDIR}/conti-otp-test
$ make
```

4. Create the installation directory

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
$ mkdir -p ${CAS_WORKDIR}/conti-otp-test_exec
```

5. "make install" command

```
$ make install DESTDIR=${CAS_WORKDIR}/conti-otp-test_exec
```

6. **Images** - Create the "external_packages" directory

```
$ mkdir -p ${CAS_WORKDIR}/external_packages
```

7. Copy the content of ${CAS_WORKDIR}/conti-otp-test_exec into ${CAS_WORKDIR}/external_packages

```
$ cp -r  ${CAS_WORKDIR}/conti-otp-test_exec/ ${CAS_WORKDIR}/external_packages
```

8. Generate a new set of images

```
$ bash ${CAS_WORKDIR}/.launchers/linxu/sdk/install-prodfs-sdk ${CAS_WORKDIR}
```

9. The new images will be in:

```
${CAS_WORKDIR}/release/image/devel/2K/
```

10. Once the images generated were flashed, the following commands can be executed to execute otp-test:
    1. **[root@mdm9x28 /root]#** tail -f /var/log/messages |grep otp-test&
    2. **[root@mdm9x28 /root]#** otp-test

    a. config locationmanager

```
dm9x28 /root]# tail -f /var/log/messages |grep otp-test&
dm9x28 /root]# otp-test
17:01:11 mdm9x28 user.debug otp-test: init_signal:[Thread Addr: 0x1a4f150] running signal handling loop
17:01:11 mdm9x28 user.info otp-test: main:Test application started
17:01:11 mdm9x28 user.info otp-test: setupConnection:Created new GIOChannel
17:01:11 mdm9x28 user.info otp-test: setupConnection:Created watch
17:01:11 mdm9x28 user.err otp-test: parseConfig:fopen /data/exchange/config.txt failed: 'No such file or directory'
17:01:11 mdm9x28 user.debug otp-test: init_method:[Thread Addr: 0x1a4f180] running method handling loop

17:01:14 mdm9x28 user.info otp-test: callbackInput:Read 1 bytes:

17:01:14 mdm9x28 user.info otp-test: callbackInput:Read 1 bytes:
locationmanager
17:01:32 mdm9x28 user.info otp-test: callbackInput:Read 23 bytes: config locationmanager
17:01:32 mdm9x28 user.info otp-test: processConfigLine:processConfigLine:locationmanager -> 0
17:01:32 mdm9x28 user.info otp-test: setupComponent:Starting LocationManager
17:01:32 mdm9x28 user.debug otp-test: tpLM_init:create location manager client proxy
17:01:32 mdm9x28 user.info otp-test: tpLM_init:successfully created location manager client proxy
17:01:32 mdm9x28 user.debug otp-test: tpSYS_createSocket:socket full name = broadcast-lm-1134
17:01:32 mdm9x28 user.debug otp-test: tpSYS_createSocket:socket full name = request-lm-1134
17:01:32 mdm9x28 user.debug otp-test: client_thread_func:[Thread Addr: 0x1a4f460] running client socket loop
17:01:32 mdm9x28 user.info otp-test: tpSYS_registerClientCallback:replacing client CB for (server-lm-642)
```

    b. locationmanager_reg-pos

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
locationmanager_reg-pos
Jul 22 17:12:54 mdm9x28 user.info otp-test: callbackInput:Read 24 bytes: locationmanager_reg-pos
Jul 22 17:12:54 mdm9x28 user.debug otp-test: update_clients_reg_list:REGISTER address [broadcast-lm-1041] scheme [LM_SCHEME_DEFAULT] signal [2]
Jul 22 17:12:54 mdm9x28 user.info otp-test: invokeAPIMethod:API call OK
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:status:                                  1 (current)
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Plugin name:                             qmi
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Aged time:                               0
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Latitude:                                20.557470 degrees
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Longitude:                               -103.461072 degrees
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Fix Type:                                1
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Horizontal speed:                        0.000000 meters/sec
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Vertical speed:                          0.000000 meters/sec
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Horizontal Dilution of Precision:        0.600000
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Vertical Dilution of Precision:          0.700000
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Position Dilution of Precision:          1.000000
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Altitude (with respect to ellipsoid):    1581.426758 meters
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Altitude (with respect to mean sea level): 1589.426758 meters
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Estimated Horizontal Position Error:     3.535534 meters
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Vertical Uncertainty:                    2.500000 meters
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:GNSS Fix Status:                         3
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Magnetic Deviation:                      6.089778
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Heading:                                 0.000000 degrees
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Magnetic Heading:                        353.910217 degrees
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Timestamp:                               1563815574800
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Number of Satellites:                    16
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position:Satellites:
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 2
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 5
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 6
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 12
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 17
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 19
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 24
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 28
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 68
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 69
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 73
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 74
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 80
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 83
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 84
Jul 22 17:12:54 mdm9x28 user.info otp-test: show_position: 85
```

c. locationmanager_dereg-pos

```
locationmanager_dereg-pos
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:status:                                  1 (current)
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Plugin name:                             qmi
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Aged time:                               0
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Latitude:                                20.557469 degrees
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Longitude:                               -103.461069 degrees
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Fix Type:                                1
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Horizontal speed:                        0.000000 meters/sec
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Vertical speed:                          0.000000 meters/sec
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Horizontal Dilution of Precision:        0.600000
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Vertical Dilution of Precision:          0.700000
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Position Dilution of Precision:          1.000000
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Altitude (with respect to ellipsoid):    1584.267578 meters
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Altitude (with respect to mean sea level): 1592.267578 meters
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Estimated Horizontal Position Error:     3.535534 meters
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Vertical Uncertainty:                    2.500000 meters
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:GNSS Fix Status:                         3
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Magnetic Deviation:                      6.089775
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Heading:                                 0.000000 degrees
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Magnetic Heading:                        353.910217 degrees
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Timestamp:                               1563815740800
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Number of Satellites:                    16
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position:Satellites:
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 2
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 5
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 6
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 12
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 17
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 19
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 24
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 28
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 68
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 69
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 73
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 74
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 80
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 83
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 84
Jul 22 17:15:40 mdm9x28 user.info otp-test: show_position: 85
Jul 22 17:15:40 mdm9x28 user.info otp-test: callbackInput:Read 26 bytes: locationmanager_dereg-pos
Jul 22 17:15:40 mdm9x28 user.debug otp-test: update_clients_reg_list:DEREGISTER address [broadcast-lm-1041] scheme [LM_SCHEME_DEFAULT] signal [2]
Jul 22 17:15:40 mdm9x28 user.info otp-test: invokeAPIMethod:API call OK
```

d. locationmanager_reg-time

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
locationmanager_reg-time
Jul 22 17:16:29 mdm9x28 user.info otp-test: callbackInput:Read 26 bytes:  locationmanager_reg-time
Jul 22 17:16:29 mdm9x28 user.debug otp-test: update_clients_reg_list:REGISTER address [broadcast-lm-1041] scheme [LM_SCHEME_DEFAULT] signal [6]
Jul 22 17:16:29 mdm9x28 user.info otp-test: invokeAPIMethod:API call OK
Jul 22 17:16:30 mdm9x28 user.info otp-test: show_time:timestamp:     1563815790
Jul 22 17:16:30 mdm9x28 user.info otp-test: show_time:year:          2019
Jul 22 17:16:30 mdm9x28 user.info otp-test: show_time:month:         7
Jul 22 17:16:30 mdm9x28 user.info otp-test: show_time:day:           22
Jul 22 17:16:30 mdm9x28 user.info otp-test: show_time:hour:          17
Jul 22 17:16:30 mdm9x28 user.info otp-test: show_time:minute:        16
Jul 22 17:16:30 mdm9x28 user.info otp-test: show_time:second:        30
Jul 22 17:16:30 mdm9x28 user.info otp-test: show_time:msecond:       1
Jul 22 17:16:30 mdm9x28 user.info otp-test: show_time:validity_bits: 1
Jul 22 17:16:31 mdm9x28 user.info otp-test: show_time:timestamp:     1563815791
Jul 22 17:16:31 mdm9x28 user.info otp-test: show_time:year:          2019
Jul 22 17:16:31 mdm9x28 user.info otp-test: show_time:month:         7
Jul 22 17:16:31 mdm9x28 user.info otp-test: show_time:day:           22
Jul 22 17:16:31 mdm9x28 user.info otp-test: show_time:hour:          17
Jul 22 17:16:31 mdm9x28 user.info otp-test: show_time:minute:        16
Jul 22 17:16:31 mdm9x28 user.info otp-test: show_time:second:        31
Jul 22 17:16:31 mdm9x28 user.info otp-test: show_time:msecond:       1
Jul 22 17:16:31 mdm9x28 user.info otp-test: show_time:validity_bits: 1
Jul 22 17:16:32 mdm9x28 user.info otp-test: show_time:timestamp:     1563815793
Jul 22 17:16:32 mdm9x28 user.info otp-test: show_time:year:          2019
Jul 22 17:16:32 mdm9x28 user.info otp-test: show_time:month:         7
Jul 22 17:16:32 mdm9x28 user.info otp-test: show_time:day:           22
```

e. locationmanager_dereg-time

```
locationmanager_dereg-timeJul 22 17:17:29 mdm9x28 user.info otp-test: show_time:timestamp:     1563815849
Jul 22 17:17:29 mdm9x28 user.info otp-test: show_time:year:          2019
Jul 22 17:17:29 mdm9x28 user.info otp-test: show_time:month:         7
Jul 22 17:17:29 mdm9x28 user.info otp-test: show_time:day:           22
Jul 22 17:17:29 mdm9x28 user.info otp-test: show_time:hour:          17
Jul 22 17:17:29 mdm9x28 user.info otp-test: show_time:minute:        17
Jul 22 17:17:29 mdm9x28 user.info otp-test: show_time:second:        29
Jul 22 17:17:29 mdm9x28 user.info otp-test: show_time:msecond:       1
Jul 22 17:17:29 mdm9x28 user.info otp-test: show_time:validity_bits: 1

Jul 22 17:17:30 mdm9x28 user.info otp-test: show_time:timestamp:     1563815850
Jul 22 17:17:30 mdm9x28 user.info otp-test: show_time:year:          2019
Jul 22 17:17:30 mdm9x28 user.info otp-test: show_time:month:         7
Jul 22 17:17:30 mdm9x28 user.info otp-test: show_time:day:           22
Jul 22 17:17:30 mdm9x28 user.info otp-test: show_time:hour:          17
Jul 22 17:17:30 mdm9x28 user.info otp-test: show_time:minute:        17
Jul 22 17:17:30 mdm9x28 user.info otp-test: show_time:second:        30
Jul 22 17:17:30 mdm9x28 user.info otp-test: show_time:msecond:       1
Jul 22 17:17:30 mdm9x28 user.info otp-test: show_time:validity_bits: 1
Jul 22 17:17:31 mdm9x28 user.info otp-test: callbackInput:Read 27 bytes: locationmanager_dereg-time
Jul 22 17:17:31 mdm9x28 user.debug otp-test: update_clients_reg_list:DEREGISTER address [broadcast-lm-1041] scheme [LM_SCHEME_DEFAULT] signal [6]
Jul 22 17:17:31 mdm9x28 user.info otp-test: invokeAPIMethod:API call OK
```

f. stop & quit

```
stop
Jul 22 17:19:15 mdm9x28 user.info otp-test: callbackInput:Read 5 bytes: stop
Jul 22 17:19:15 mdm9x28 user.info otp-test: processSpecialCall:Stopping all periodic commands
quit
[root@mdm9x28 /root]# Jul 22 17:19:39 mdm9x28 user.info otp-test: callbackInput:Read 5 bytes: quit
Jul 22 17:19:39 mdm9x28 user.info otp-test: main:Exiting
```

3. **[root@mdm9x28 /root]**# otp-test –help

```
OTP Test Application

Usage:
        otp-test        Running without parameters will execute commands from /data/exchange/config.txt
        otp-test help [[component_name [special_function_name]] Prints help, with detailed info if "component_name" specified

Available commands:
  <component_name>_<method_name> <args...>      Calls a method of the specified component (if it has been initialised)
  config <component_names..>    Initializes specified test components
  config        Initializes all available test components
  register <time_interval> <max_count> <component_name>_<callback_name> Registers a callback and logs an error if it executed more than "max_count" in "time_interval" seconds
  nullcheck <null_positions..> <component_name>_<method_name> <args..>  Tests "method name" with NULL parameters at specified positions
  periodic <timeout_sec> <component_name>_<method_name> <args..>        Registers a periodic call of "method_name"
  stop  Stops all periodic calls
  help [[component_name [special_function_name]]        Prints help, with detailed info if "component_name" specified
  quit  Exits application
```

3.1.7.5 Adding a Service [ALL]

1. Verify the following configuration in your service file:

```
[Unit]
Description=Eso-Hook
After=level3.target data.mount
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
[Service]
Type=simple
Restart=always
ExecStart=/bin/sh /opt/eso/conmod_mount.sh

[Install]
WantedBy=multi-user.target
```

2. For the service to start automatically during boot, when the service file is in /lib/systemd/system, create a soft link to the *.service under one of the following:

Path	Purpose
level1.target.wants	Hi-prio, required for E-Call
level2.target.wants	Medium prio
level3.target.wants	Low priority, engineering debug tools

3.1.8 Testing/Error Reporting

Not applicable

3.2 Linux Kernel using SDK

This section explains how to enable compilation of linux source code, adding continental's patches, and kernel module compilation using the SDK environment.

This enables the user to:

- Download Kernel Source Code
- Add custom .dts to Kernel Compilation
- Build Kernel Modules based on defined Kernel
- Load modules into the target

On the other hand, this does not enable user to flash a different Kernel into the target

File provided kernel_amend.tar.gz has the following contents

- .config
- audi-conmod.dts
- linux_<< LOCALVERSION >>_patches.tar.gz
- local_version

3.2.1 Compiling the Kernel source code

3.2.1.1 Downloading Linux Source Code

In your SDK environment, after setting up local variables, download the source code that matches the given Linux Kernel version from codeaurora. Untar the kernel_amend.tar.gz file and source local_version file.

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
cd ${CAS_WORKDIR}
mkdir -p ${CAS_WORKDIR}/tmp
tar -xvzf kernel_amend.tar.gz -C tmp
source tmp/local_version
git clone -b ${KERNEL_TAG} https://source.codeaurora.org/quic/la/kernel/msm-4.14/
```

3.2.1.2 Applying conti patches to Kernel

Decompress linux_${LOCALVERSION}_patches.tar.gz file, inside msm-4.14 folder and apply the patches.

```
mv ${CAS_WORKDIR}/tmp/* ${CAS_WORKDIR}/msm-4.14
cd ${CAS_WORKDIR}/msm-4.14
tar -xvzf linux_${LOCALVERSION}_patches.tar.gz
for i in linux_patches/*.patch; do echo "$i"; patch -p1 < $i; done
cp -f  audi-conmod.dts arch/arm/boot/dts/qcom/cas-audi-conmod.dts
```

3.2.1.3 Compilation

Make sure LOCALVERSION variable is available by source to local_version file. Also make sure .config was copied into msm-4.14, variables are set.

```
mv ../tmp/.config ./
export ARCH=arm
export CROSS_COMPILE=${CROSS}
make silentoldconfig
make CFLAGS_KERNEL="-DUTS_NODENAME=\\\"sa515m\\\" -Wno-format -Wno-misleading-indentation -Wno-int-to-pointer-cast -Wno-pointer-to-int-cast -Wno-bool-operation -Wno-unused-value -Wno-uninitialized -Wno-enum-compare  -Wno-parentheses -Wframe-larger-than=1080 -Idrivers/platform/msm/ipa/ipa_v3 -Idrivers/platform/msm/ipa/ipa_clients" dtb-y="qcom/cas-audi-conmod.dtb" KBUILD_IMAGE=zImage-dtb
```

3.2.2 Kernel Module Compilation

After having the Kernel source code compiled (Section 3.2.1), a Kernel module can be compiled.

Next instructions detail how to create a Kernel Module compiled using the Kernel Source Code (Section 3.2.1) and how to load the generated module into the target.

3.2.2.1 Hello World Kernel Module Example

3.2.2.1.1 Preconditions

- Have an SDK workarea

© Continental Automotive GmbH 2019
Confidential and Proprietary

- Setup the environment variables (Refer to section 3.1.2.2)

```
export ARCH=arm
export CROSS_COMPILE=${CROSS}
```

- Compile a Kernel source code (Refer to section 3.2.1)

3.2.2.1.2 Source files hello_world.c and Makefile

Create a folder inside your SDK workarea and create the following files.

```
mkdir -p ${CAS_WORKDIR}/ext_driver/hello_world && cd ${CAS_WORKDIR}/ext_driver/hello_world
```

hello_world.c

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/printk.h>

MODULE_LICENSE("Dual BSD/GPL");

static int hello_init(void)
{
  printk(KERN_ALERT "Hello, world\n");
  return 0;
}

static void hello_exit(void)
{
  printk(KERN_ALERT "Goodbye, cruel world\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

Makefile

© Continental Automotive GmbH 2019
Confidential and Proprietary

```
ifneq ($(KERNELRELEASE),)
    obj-m := hello_world.o
else
    obj-m := hello_world.o
    KERNEL_SRC ?= ${CAS_WORKDIR}/msm-4.14/
    PWD := $(shell pwd)
default:
    $(MAKE) -C $(KERNEL_SRC) M=$(PWD)
endif
```

3.2.2.1.3 Module Compilation

Add into the make command the KERNEL_SRC variable with the path where the Linux source code was downloaded and compiled.

```
cd  ${CAS_WORKDIR}/ext_driver/hello_world
make  KERNEL_SRC=${CAS_WORKDIR}/msm-4.14/ ARCH=arm
```

3.2.2.1.4 Generate images including the module

Create folder structure for external_packages folder, copy the ".ko" file resulting from the compilation into this folder. Generate images.

```
mkdir  -p ${CAS_WORKDIR}/external_packages/lib/modules/4.9/extra
cp  ${CAS_WORKDIR}/ext_driver/hello_world/hello_world.ko ${CAS_WORKDIR}/external_packages/lib/modules/4.9/extra/
cd ${CAS_WORKDIR}
bash .launchers/linux/sdk/install-prodfs-sdk --workspace ${CAS_WORKDIR} --image_size <image_size>
[--signed_images  <path to conti-signed-images file>]  [--build_version <build_vers>]
```

3.2.2.1.5 Load the module

In the target after flashing the image, load the module with "insmod"

```
[root@sa515m ~]# insmod /lib/modules/4.9/extra/hello_world.ko
[ 285.693443] Hello, world
[root@sa515m ~]# rmmod hello_world
[  309.421876] Goodbye, cruel world
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

3.2.3 Flash new Kernel into target

This section provides a set of instructions to have the downloaded kernel to be flashed into the target. In this guide a change in the kernel configuration is performed.

3.2.3.1 Prerequisites

- Follow instructions to have the Kernel compilation using the SDK (Section Compiling the Kernel source code)
- Have all the variables correctly set in your environment
- Have sourced local_version file

3.2.3.2 Perform required changes on Kernel

Perform the changes in the kernel, this guide mentions how to enter menuconfig environment. **Changes in this configuration may affect the functionality.** Perform the changes as needed.

```
cd ${CAS_WORKDIR}/msm-4.14
make menuconfig
## Make the changes and exit the environment
```

${CAS_WORKDIR}/msm-4.14/**.config** file is updated, it is required to perform another compilation to update. Some warnings may be now visible in the compilation.

3.2.3.3 Generate new boot.img

The following command sets some properties over the Kernel command line, this is the configuration used for TP release, if modified, logs will change on the target. If user changes the compression algorithm, then the command needs to be adapted.

```
cd ${CAS_WORKDIR}/msm-4.14
mkbootimg --kernel ./arch/arm/boot/zImage-dtb --ramdisk /dev/null --cmdline "console=ttyMSM0,115200,n8 noinitrd loglevel=4 ubi.mtd=? rootfstype=ubifs ro systemd.show_status=false msm_rtb.filter=0x37 lpm_levels.sleep_disabled=1 service_locator.enable=1 """ --pagesize 4096 --base 0x80000000 --tags-addr 0x82000000 --ramdisk_offset 0x0 --output boot_4096.img
```

This command will create a boot_4096.img file inside ${CAS_WORKDIR}/msm-4.14 directory.

3.2.3.4 Flash into target

3.2.3.4.1 Using QFil

- Generate the set of images
- Substitute boot.img from ${CAS_WORKDIR}/release/images/devel/4K/ with the boot_4096.img from section Generate new boot.img,
- Rename it to boot.img.
- Flash normally.

3.2.3.4.2 Using Fastboot

- Generate the set of images

© Continental Automotive GmbH 2019
Confidential and Proprietary

- Send the board to bootloader
- Substitute boot.img from ${CAS_WORKDIR}/release/images/devel/4K/ with the boot_4096.img from section Generate new boot.img,
- Rename it to boot.img.
- Flash boot partition
- Reboot board

```
[root@sa515m ~]# reboot-bootloader
# From cmd
fastboot flash boot boot.img
fastboot reboot
```

3.2.3.5 Warnings

- Changes performed on the kernel may affect functionality on the board, **changes are under the user responsibility.**
- If using a different LOCALVERSION than the one provided in kernel_amend.tar.gz/local_version file results on
  - Modules not being loaded.
  - Services not starting correctly
- Testing over this feature is minimal in TP side
- Changing the .dts files and flashing the outcome kernel into the target has not been tested by TP.
- Changing mkbootimg command and its properties may change the functionality on the target.
- Do not use a Kernel_amend.tar.gz different from the one provided in version installed in the target.

3.3 Increase system partition size.

This section describes one of the approaches to resize system partition. Current approach use python script **resize_system.py** which is running on the target and change system partition size in run time.

3.3.1 Run resize_system.py script.

Scrip **resize_system.py** located in the SDK folder. In order to get arguments which can be use with the script need to run it with arg “-h” or without arguments:

```
D:\conti\main\sa515m\scripts> C:\Python311\python.exe .\resize_system.py

Invalid argument provided. Please see the help below:
usage: resize_system [option]

options:
  -h, --help          show this help message and exit
  -s SIZE, --size SIZE  Set new system partition size in MiB
  -c                  Show current size of system partition
  -a                  Show available space of UBI 0
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

If need to check current available size in the target in which will be able increase system partition can be used option '-a'. For checking current size of using system partition use option '-c'. In order to reseze current system partition use opetion '-s' or '--size' with value which points to the new size of partition. If script run without arguments or with incorrect ones script print help information.

After script finishing system partition be mounted like rw, but if target reboot in order to update files in system partition need to remount it:

```
mount -o remount,rw /
```

3.3.2 Warninig

If new size less then current, script print current and new one and quit.

If increase of size is bigger than available ubi space, script print max size in which partition can be increase and quit.

3.3.3 Prerequisite

Connect chestnut or TCU to the host.

Check connection to target throw adb.

3.3.4 Additional software

Install python version 3.11 or higher.

Install adb tool.

Set path to adb tool in the system variables.

4 Known Issues

Warnings are expected during SDK constructor execution.

5 FAQs

Not applicable

6 Troubleshooting/Tips and Tricks

Not applicable

7 References

7.1 Abbreviations/Acronyms/Definitions

Not applicable

7.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary

© Continental Automotive GmbH 2019
Confidential and Proprietary

TP_F101U-00054: SecureBoot 5G NAD ConMod User Guide

7.3 Website links

Not applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

8 Revision History

Version	Date	Description
00.04	Aug 19, 2019	Initial release
00.18	Jan 21, 2021	Add description to install signed images - " **conti-signed-images.tar.gz**" - to workarea
00.19	Mar 08, 2021	Add description of SDK feature "UNECE": ConMod "Support of Integrity Validation Data"
00.20	Mar 30, 2021	Add description of feature "Suspend Secure Boot"
		== *ConMod-specific content from here* ==
01.00	Nov 30, 2021	• Modified interface of "install-prodfs-sdk": named parameters • Add log file name "summary.log" to description of log location.
02.00	Aug 21, 2024	Add section 3.1.6. Feature "Sign custom Trusted Application"
03.00	Nov 11, 2024	Add chapter “Increase system partition size”

© Continental Automotive GmbH 2019
Confidential and Proprietary